Data Structure and Programming Spring 2025
Written Assignment 3

Chiao-Yun, Chin b10901154@ntu.edu.tw
Yi-Chen, Wang b10901143@ntu.edu.tw
Yi-Chou, Lin b10901149@ntu.edu.tw

Release Date: 12:00 on 04/10/2025
Due Date: 23:59 on 05/01/2025
Late policy: Score = Score $\times$ $0.7^{(h/24)}$, h = late hours ≤ 48

1. **(20 pts) Splay tree**

   Given the initial splay tree in Figure 1, perform each of the following operations independently. That is, for each subproblem, always start from the original splay tree in Figure 1, perform the specified operation, and analyze the transformation separately.

   Draw all the intermediate splay tree and indicate the number of zig-zig, zig-zag, and simple rotations performed.

   (a) (7 pts) Insert(20) using top-down splaying from figure 1.
   (b) (7 pts) Delete(5) using bottom-up splaying from figure 1.
   (c) (6 pts) Insert(9) using bottom-up splaying from figure 1.

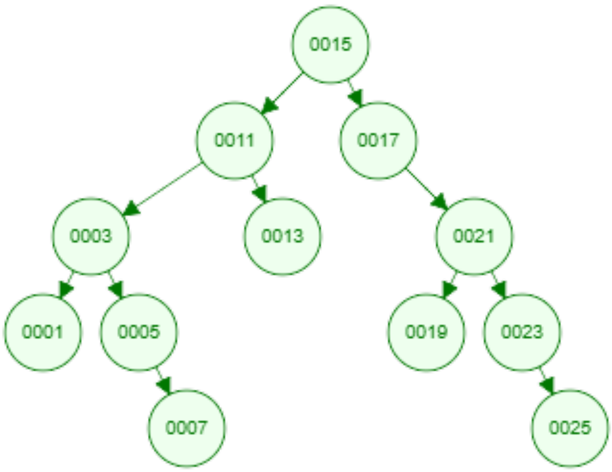

Figure 1: Initial Splay Tree

2. **(10 pts) Time complexity of AVL tree and Splay tree**

   Fill in the time complexity of search in the table below. The time complexity should be expressed in terms of n, where n is the number of items in the tree. You can use big-O notation.

Search	Binary Search Tree	Splay Tree	AVL Tree
Worst case			
Average	O(n)		

Table 1: Time complexity of search in different trees

3. **(20 pts) Data Structure for Different Scenarios**

   For each of the following scenarios, choose the most suitable data structure from the options below and explain why it is the best choice compared to the others.

   Options:

   - Stack
   - Queue
   - AVL tree
   - Splay tree

   (a) (5 pts) A search system frequently accesses a few popular keywords, while most others are rarely searched.

   (b) (5 pts) A print server processes print jobs in the order they are received.

   (c) (5 pts) A database needs to maintain a large, dynamically changing set of records while ensuring fast search, insertion, and deletion operations, even in the worst case.

   (d) (5 pts) A text editor provides an "Undo" function that allows users to revert the most recent changes first.

4. **(15 pts) AVL Tree**

   Prove that whenever a node in an AVL tree becomes unbalanced due to an insertion, it can be balanced by single or double rotation at most once. You can draw the trees and indicate the height to illustrate your proof.

5. **(15 pts) Amortized Analysis**

   We have designed a stack that supports a MultiPop operation, with the following functionalities:

   - Push(x): Pushes the element x onto the stack.
   - Pop(): Pops the top element from the stack.
   - MultiPop(k): Pops k elements from the stack. If there are fewer than k elements in the stack, it pops all of them.

   The time complexity of Push and Pop is O(1).

   (a) (5 pts) What is the worst-case time complexity of MultiPop? Assume that there are n elements in the stack. Please explain your answer.

   (b) (10 pts) Start from empty stack. Given a sequence of n operations, including Push, Pop, and MultiPop, please analyze the amortized time complexity of an operation in the sequence.

6. **(20 pts) Algorithm Design**

   Design an algorithm to find the successor of an element in the binary search tree. The successor of an element x is the smallest element in the tree that is greater than x. The time complexity of the algorithm should be O(h). h represents the height of the tree. (You can write in pseudo code, python or c++.)